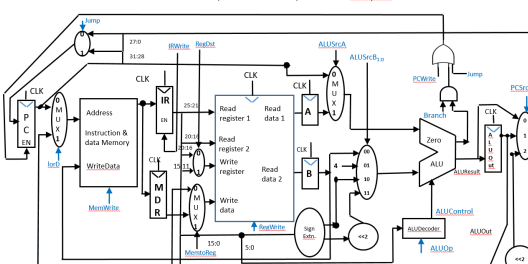
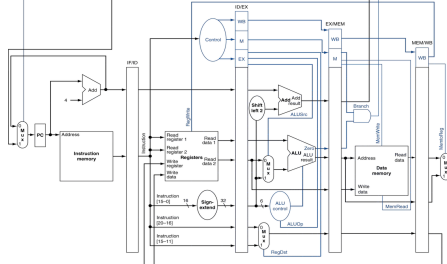


Single-cycle (exam fig): M1 Jump (left), M2 RegDst, M3 ALUSrc, M4 PCSrc (top), M5 MemtoReg; A1 = PC+4, A2 = branch adder.



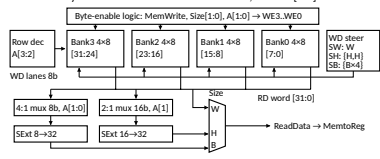
Multicycle (exam fig): M0 IorD, M1 Jump, M2 RegDst, M3 MemtoReg, M4 ALUSrcA, M5 ALUSrcB, M6 PCSrc; non-arch regs IR, MDR, A, B, ALUOut.



Pipelined datapath + control (slide 32; P&H: control in ID → WB/M/EX fields in ID/EX [96], EX/MEM (5b), MEM/WB (2b)).

#38 Memory size & memory unit for SW/SH/SB/LW/LH/LB 24-3c

Size: 32-bit byte address $\Rightarrow 2^{32}$ bytes = 4 GB = 2^{30} words $\times 32$ bits. Organise as 4 byte-wide banks ($2^{30}/8$ decoders). Reads: LW = word; LH = 32-bit half word. For 16 byte locations: 4 banks $\times$ 4 rows $\times$ 8 b, row = A[3:2].

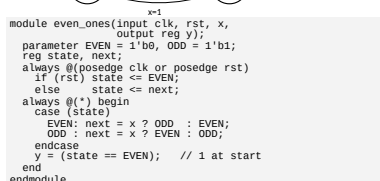


WE[3:0] (MemWrite[1:0], SW → 1111; SH → A[1]-0; 0011; A[1]-1; 1100; SB → one-hot of A[1:0] 24 decoder). Reads: LW = word; LH = 32-bit half word. For 16 byte locations: 4 banks $\times$ 4 rows $\times$ 8 b, row = A[3:2].

Components & sizes (full 32-bit): 4 banks of $2^{30}/8$; row decoder $2^{30}/2^{30}$ 24 decoder & gates for WE; write-strobe 3:1 mux 32 b; read: 4:1 mux 8 b, 2:1 mux 16 b, 2:1 mux 8×32 & 16-32; 3:1 mux 32 b. (16-loc version: 4 banks 4-8, 2-4 row decoder).

#39 Even number of 15 detector + testbench (Verilog) 23-4

Moore FSM, 2 states: **EVEN** (out 1, reset/start state) and **ODD** (out 0); input 1 toggles state, input 0 keeps it. So output = 1 at the start (zero is even).



module tb;
 reg clk = 0, rst = 1, x = 0; wire y;
 even_ones out(clk, rst, x, y);
 initial begin
 \$monitor("Not x=xb", \$time, x, y);
 #12 rst = 0;
 x = 1; #10 x = 0; #10 x = 1; #10 x = 1;
 #10 x = 0; #10 x = 1; #10 \$finish;
 end
endmodule

#28 NOPs, forwarding+speed-up, only scheduling 23-6

(a) No fwd: (n+0.4n)/250 = 350n ps. Fwd: (n+0.05n)/300 = 315n ps. Speed-up = 350/315 = 1.11.

(b) Lat = NOP fraction without forwarding (forwarded version still 0.05n). Need (1+250) = 1.05-300 + 315 = 1+1.26 $\Rightarrow$ x $\times$ 26% NOPs. (If NOPs scale proportionally w/8 with fwd then $\sim$ 300(1+0.8) $\Rightarrow$ x $\times$ 23.5%.)

(c) Assume RF written 1st half / read 2nd half $\Rightarrow$ dependent instr must be $\geq$ 3 slots after producer (2 NOPs if adjacent).

(1) with NOPs	(4) best schedule
add \$s3, \$s1, \$s9	add \$s3, \$s1, \$s9
nop	lw \$s1, 0(\$s4)
lw \$s3, 4(\$s3)	nop
lw \$s1, 0(\$s4)	lw \$s3, 4(\$s3)
nop	nop
or \$s2, \$s3, \$s2	or \$s2, \$s3, \$s2
nop	nop
sw \$s2, 0(\$s3) ; 5 NOP	sw \$s2, 0(\$s3) ; 5 NOP

add → lw → or → sw is a new RAW chain, each needs 3 slots $\Rightarrow$ sw at slot $\geq$ 10 $\Rightarrow$ 10 slots - 5 instrs = 5 NOPs minimum. Only scheduling 23-6

smaller is faster; good design demands good compromises. MIPS got so good it can't move above add (WAR; add reads \$s1). So NOPs cannot go below 5 without forwarding (with forwarding: 0 NOPs needed).

D. REFERENCE — slides + Harris/P&H

R1 Instruction formats & 3 addressing modes

Type	31:26	25:21	20:16	15:11	10:6	5:0
R	op[6:0]	rs	rt	rd	shamt	funct
I	op	rs	rt	rd	shamt	funct
J	op	address[26]				

1. Register: all operands in regs — add \$m0,\$s1,\$s2

2. Immediate: 16-bit constant in instr (sign-ext) — add \$s0,\$s1,5

3. Base (displacement): addr = RF[rs] + SignExt(imm) — lw \$t0,\$s1(\$s1)

4. PC-relative: target = PC+4 + SignExt(imm) — beq/bne

5. Pseudo-branch: target = PC+4[31:28], add26, 00) — j, jal

Design principles: simplicity favours regularity; make the common case fast; smaller is faster; good design demands good compromises. MIPS got so good it can't move above add (WAR; add reads \$s1). So NOPs cannot go below 5 without forwarding (with forwarding: 0 NOPs needed).

R2 Opcodes / function (hex)

Instr	op	Jump	RdSt	RW	ASrc	Br	ALUOp	Mrd	MWr	MZr
add	000000	0	1	0	0	0	10	0	0	0
sub	000000	0	1	0	0	0	10	0	0	0
and/or	000000	0	1	0	0	0	10	0	0	0
sll	000000	0	1	0	0	0	10	0	0	0
sll/srl	000000	0	1	0	0	0	10	0	0	0
sra	000000	0	1	0	0	0	10	0	0	0
lw/sw	100011 1023 / 101011 28	0	1	0	0	0	10	0	0	0
beq/bne	000100 004 / 000101 05	0	1	0	0	0	10	0	0	0
addi/slli	000100 008 / 000101 05	0	1	0	0	0	10	0	0	0
andi/ori	000100 001 / 000101 05	0	1	0	0	0	10	0	0	0
lwl/lwr	000100 001 / 000101 05	0	1	0	0	0	10	0	0	0
lwl/lwr	000100 001 / 000101 05	0	1	0	0	0	10	0	0	0

Regs: 0 zero | \$1 \$at | \$2-3 \$vo-1 | \$4-7 \$ao-3 | \$8-15 \$to-7 | \$16-23 \$so-7 | \$24-25 \$t8-9 | \$26-27 \$k0-1 | \$28 \$gp | \$29 \$sp | \$30 \$fp | \$31 \$ra.

R3 Main decoder table (single-cycle; code only slide 25)

RdSt=RegDst, KWr=RegWrite, ASrc=ALUSrc, Br=Branch, MRd/MWr=MemRead/Write, M2R=MemtoReg, RegDSt 0=rt, 1=rd; ALUSrc 0=RD2, 1=SignImm; MemtoReg 0=ALUResult, 1=ReadData; PCSrc = Branch.Zero (bne: Branch--Zero); Jump selects JTA. (Pipelined table slide 29 shows sw MemRead=1 — use 0.)
